PROYECTO INTEGRADOR · DESARROLLO MÓVIL CON ANDROID

Gota de Vida

Una app que conecta a hospitales de la Comarca Lagunera con donadores de sangre compatibles y cercanos cuando hay una urgencia.

QUÉ ES ESTE DOCUMENTO

La explicación del proyecto para el equipo

REGIÓN

Torreón, Gómez Palacio y Lerdo

META

115 de 115 puntos

1 El problema

Cuando un hospital necesita sangre con urgencia, sobre todo de un tipo poco común como **O negativo**, hoy depende de publicaciones en Facebook, cadenas de WhatsApp y de que los familiares del paciente consigan donadores por su cuenta.

Eso tiene tres fallas:

- **Se pierde tiempo:** el mensaje tarda horas en llegar a la persona correcta.
- **Llega a quien no sirve:** se contacta a gente que no es compatible o que donó hace poco y todavía no puede volver a donar.
- **No hay seguimiento:** el hospital no sabe cuántos van en camino ni si ya cubrió las unidades que necesita.

> **Nuestra idea en una frase:** que la urgencia le llegue *solo* a las personas que sí pueden ayudar, en minutos, y que puedan agendar su cita con un mensaje.

2 Cómo funciona

1

El donador se registra con su tipo de sangre, sus datos para saber si puede donar y su ubicación aproximada.

2

El hospital publica una urgencia: "Urge O−, 3 unidades, Hospital Universitario".

3

La app filtra a los donadores compatibles, cercanos y que ya pueden volver a donar, y les avisa por notificación y WhatsApp.

4

El donador acepta y agenda su cita, en la app o platicando con el chatbot por WhatsApp.

5

En el hospital escanean el código QR del donador y la donación queda confirmada.

6

La app actualiza el historial y le dice al donador cuándo podrá volver a donar.

3 Quién la usa y qué ve

DONADOR

- Registro y perfil
- "Puedes donar" o "podrás en N días"
- Urgencias cercanas en el mapa
- Agendar cita y su código QR
- Historial e insignias
- Chat con el asistente

HOSPITAL

- Publicar una urgencia
- Ver en vivo cuántas unidades van cubiertas ("2 de 3")
- Escanear el QR del donador
- Estadísticas del banco de sangre

ADMINISTRADOR

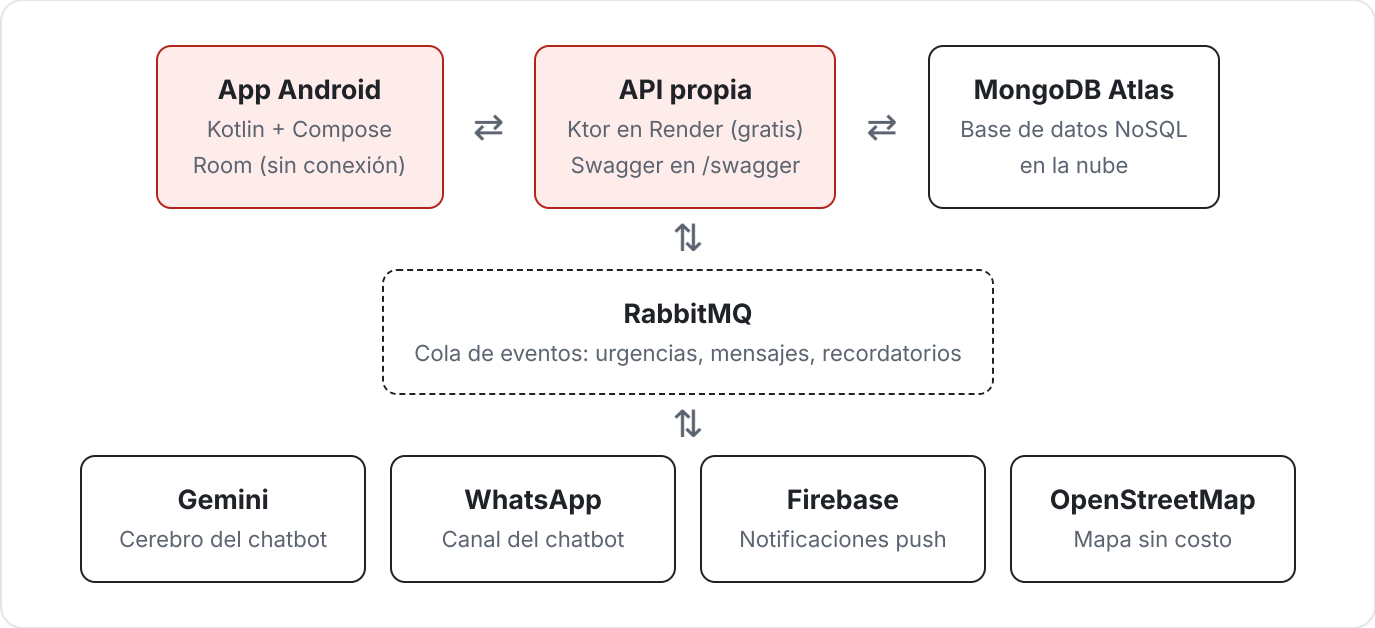
- Dar de alta hospitales
- Editar los requisitos para donar sin tocar el código

4 Cómo cumplimos la rúbrica

Criterio	Cómo lo cumplimos	Puntos
Arquitectura y lenguaje	App 100% Kotlin con Jetpack Compose. Carpetas por funcionalidad con las capas UI, ViewModel, Service, Repository y Entity (Screaming Architecture).	**15**
Problema real	Urgencias de sangre en La Laguna.	**15**
API en la nube + Swagger	Backend propio en Ktor (también Kotlin), desplegado gratis en Render, con Swagger documentando todos los endpoints y una colección de Postman con pruebas automáticas.	**15**
Base de datos en la nube	MongoDB Atlas (plan gratuito), con búsqueda de donadores por cercanía.	**10**
Servicios de terceros	Gemini (IA), WhatsApp de Meta, Firebase (notificaciones), login con Google y mapa de OpenStreetMap.	**15**
Git y colaboración	Ramas `feature/*`, Pull Requests revisados, `main` protegida y commits de los tres.	**15**
Exposición	5 minutos de diapositivas: problema, solución y arquitectura.	**5**
Demo en vivo	15 minutos con tres teléfonos siguiendo un guion ensayado (sección 11).	**5**

Vestimenta	Los tres con ropa formal.	**5**
Extra: NoSQL	MongoDB es NoSQL y está en el centro de la lógica.	**+3**
Extra: Colas de mensajes	RabbitMQ reparte las urgencias, los mensajes de WhatsApp y los recordatorios.	**+3**
Extra: Vibecoding	Prompt maestro + bitácora de cada prompt que usemos.	**+3**
Extra: Room	La app funciona sin internet y sincroniza al volver la señal.	**+3**
Extra: CI/CD e idiomas	GitHub Actions compila y prueba cada PR; la app está en español e inglés.	**+3**
Total posible		**115**

5 Cómo se conecta todo



La llave de Gemini y la de WhatsApp viven solo en el backend, nunca dentro de la app. Así nadie puede sacarlas descompilando el APK.

6 Cómo se organiza el código

La rúbrica pide **Screaming Architecture**: que al abrir el proyecto las carpetas "griten" de qué trata la app (donación de sangre), no qué tecnología usa. Por eso organizamos por funcionalidad, y dentro de cada una van las mismas cinco capas.

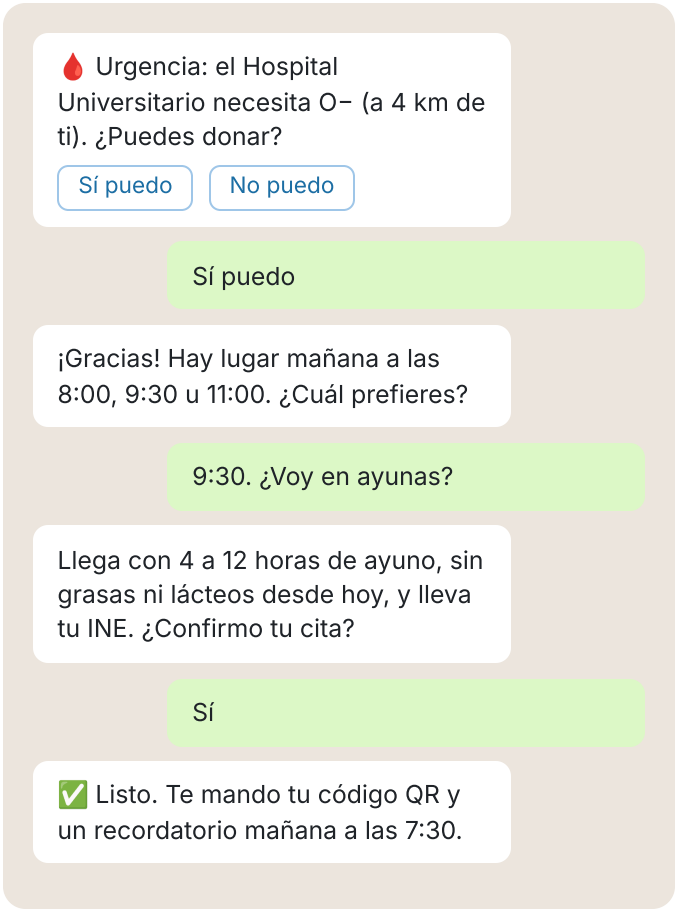
```
mx.gotadevida/
├── core/          (lo compartido)
├── autenticacion/
├── donador/
├── solicitud/     (urgencias)
├── cita/
├── historial/
├── chatbot/
├── hospital/
└── notificaciones/
```

```
solicitud/
├── ui/          pantallas Compose
├── viewmodel/   controla el estado
├── service/     reglas de negocio
├── repository/  API + Room
└── entity/      modelos de datos
```

> **Regla de oro:** la pantalla solo habla con su ViewModel, el ViewModel con el Service o el Repository, y el Repository con la API y con Room. Nunca nos saltamos capas. El backend usa la misma estructura.

7 El chatbot "Gota"

No es un bot que solo contesta preguntas: **hace cosas**. Gemini puede llamar funciones de nuestra API para consultar si el donador puede donar, ver horarios y agendar o cancelar citas. Funciona igual dentro de la app y por WhatsApp.



Reglas del bot

- Responde los requisitos con los datos de nuestra base, nunca inventados.
- Nunca dice que alguien "sí puede donar": eso lo decide el médico del banco de sangre.
- Pide confirmación antes de agendar o cancelar.
- Mensajes cortos, en el idioma del usuario.

Ojo con WhatsApp

- Para que el bot escriba primero, Meta tiene que aprobar una plantilla de mensaje.
- En modo de prueba solo manda mensajes a unos pocos números verificados: registramos los nuestros y el del profe.

8 Lo que nos hace destacar

Avisos por oleadas

Primero avisamos a los 20 donadores más cercanos. Si en 15 minutos no se cubren las unidades, la app amplía el radio y avisa a más gente.

Contador en vivo

El hospital ve cómo sube "1 de 3… 2 de 3" conforme los donadores aceptan.

Requisitos editables

En octubre de 2026 salió una nueva norma (NOM-253-SSA1-2025) que manda los requisitos a una guía que puede cambiar. Por eso los guardamos en la base de datos y no en el código.

Funciona sin internet

Si no hay señal, la acción se guarda y se envía sola cuando vuelve la conexión.

QR en el hospital

Al escanearlo se confirma la donación y se calcula la próxima fecha en que se puede donar.

Detalles que suman

Widget con los días que faltan para volver a donar, insignias, agregar la cita al calendario y abrir la ruta al hospital en la app de mapas.

Privacidad: el hospital nunca ve dónde vive el donador. Solo ve la distancia aproximada y, cuando el donador acepta, su nombre y teléfono.

9 Reparto de trabajo

Propuesta para que los tres tengan commits reales y bien repartidos. Todos revisamos los Pull Requests de los demás.

Integrante 1

Backend y nube

- API en Ktor
- MongoDB Atlas
- Swagger y colección de Postman
- Despliegue en Render
- Seguridad (login, roles)

Integrante 2

App del donador

- Pantallas en Compose
- Room y modo sin conexión
- Mapa y QR
- Widget
- Español e inglés

Integrante 3

Integraciones

- RabbitMQ
- Notificaciones push
- Chatbot con Gemini
- WhatsApp
- GitHub Actions y panel del hospital

10 Costo cero

Todo el prototipo usa planes gratuitos que no piden tarjeta. Lo único que hay que cuidar son sus límites:

Servicio	Límite que nos afecta
Render (backend)	Se duerme tras 15 min sin uso y tarda cerca de 1 min en despertar: lo abrimos antes de la demo. No da workers gratis, así que la cola se procesa dentro del mismo backend.
MongoDB Atlas M0	0.5 GB de datos, más que suficiente para el prototipo.
CloudAMQP (RabbitMQ)	20 conexiones y 1 millón de mensajes al mes: usamos una sola conexión.

Gemini	Pocas peticiones diarias y Google puede leer los mensajes del plan gratis: respondemos lo frecuente sin IA y nunca mandamos datos personales.
WhatsApp	Número de prueba gratis, solo hacia pocos números verificados (los nuestros y el del profe).
Mapas	Google Maps pide tarjeta, así que usamos OpenStreetMap.
Postman	El plan gratis es para una sola persona: guardamos la colección en el repo y cada quien la importa en su cuenta. En GitHub Actions corre con Newman, que es gratis.
GitHub	Actions gratis; con el Student Pack tenemos Copilot y GitHub Pro sin costo.

11 Plan de trabajo

#	Fase	Lista cuando...
A	Diseño en Claude Design	Los tres aprobamos el sistema de diseño y las 11 pantallas.
B	Repositorio	Repo creado, `main` protegida, diseños en `docs/design/` y un PR de prueba pasa el CI.
1	Backend base	Swagger público funcionando y conectado a MongoDB Atlas.
2	App base	Nos registramos desde el teléfono de punta a punta.
3	Urgencias	El hospital publica y solo los donadores compatibles reciben la notificación.
4	Citas	Flujo completo hasta la donación confirmada con QR.
5	Chatbot	Agendamos una cita platicando por WhatsApp.
6	Pulido	Modo sin conexión, widget, insignias y datos de demo.
7	Presentación	Diapositivas y demo ensayadas con cronómetro.

Guion de la demo (15 minutos, 3 teléfonos)

Min	Qué mostramos
1	Teléfono A es el hospital; B es donador O−; C es donador A+.
2	Registro en vivo de un donador nuevo.

3	El hospital publica "Urge O−". **Solo B recibe el aviso; C no, porque no es compatible.** Esta es la parte clave.
3	B agenda la cita platicando con el bot por WhatsApp; el contador del hospital sube en vivo.
2	C activa el modo avión, acepta una urgencia, quita el modo avión y se sincroniza.
2	El hospital escanea el QR de B y la donación queda confirmada.
2	Detrás de escena: Swagger, el flujo completo en Postman con las pruebas en verde, los eventos en RabbitMQ, un PR con CI en verde y la app en inglés.

12 Qué sigue

- [ ] Confirmar el reparto de trabajo y quién hace qué.
- [ ] Crear el repositorio en GitHub, agregarnos a los tres y proteger `main`.
- [ ] Diseñar el sistema de diseño y las pantallas en Claude Design.
- [ ] Crear cuentas gratuitas, sin tarjeta: MongoDB Atlas, CloudAMQP (RabbitMQ), Render, Firebase, Google AI Studio (Gemini) y Meta for Developers (WhatsApp).
- [ ] Activar el GitHub Student Developer Pack (Copilot gratis y GitHub Pro).
- [ ] Copiar el prompt maestro (`docs/ai/PROMPT_MAESTRO.md`) al repo como `CLAUDE.md` y `.github/copilot-instructions.md`.
- [ ] Preguntarle al profe si usar Claude cuenta para el punto de vibecoding (la rúbrica menciona Cursor y Copilot).
- [ ] Confirmar los requisitos para donar con un banco de sangre real de La Laguna.
- [ ] Preguntarle al profe si quiere las APIs probadas en Postman (de todos modos la vamos a incluir).

El detalle técnico completo (endpoints, eventos de la cola, funciones del chatbot y reglas de trabajo para la IA) está en `docs/ai/PROMPT_MAESTRO.md`.